Training-Efficient Physics-Informed Neural Networks for Incompressible Laminar Flow Around a Cylinder

Chi-Han Chiu, Chen-Yan Juang, Ming-Yang Wu, and Jing-Hua Chang (Team 12)
Department of Electrical and Computer Engineering
University of California San Diego
{c8chiu, chjuang, miw090, jic184}@ucsd.edu

Abstract

Physics-informed neural networks (PINNs) provide a mesh-free way to learn continuous flow fields while enforcing governing equations through residual losses. We reproduce and extend the mixed-variable PINN of Rao et al. for two-dimensional incompressible laminar flow around a cylinder at $Re = 10$. Against a Chorin projection reference solution, our reproduction obtains post-initial-transient mean relative L_2 errors of 5.30%, 19.86%, and 10.79% for u, v, and p, respectively. We then study training-side directions: a GPU-resident L-BFGS backend, residual-aware adaptive collocation, and causal loss scheduling. The GPU-resident backend achieves a $1.23\times$ speedup and 11.0% lower estimated energy while maintaining comparable accuracy. Causality-weighted Adam pretraining followed by unweighted L-BFGS polish gives the best tested post-initial-transient mean relative L_2 errors of 4.64%, 18.60%, and 8.89%. Overall, the completed quantitative results indicate that optimizer implementation and loss scheduling are critical to PINN efficiency and stability; adaptive collocation remains a promising sampling direction that should be validated with fixed-grid error metrics. We additionally outline two exploratory directions, alternative network architectures and a three-dimensional extension, as preliminary work.

1 Introduction

Fluid dynamics is governed by nonlinear partial differential equations whose solutions are central to engineering design, aerodynamics, biomedical devices, and industrial systems. Traditional numerical solvers such as finite-volume and finite-element methods remain the standard tools for high-fidelity simulation. However, they often require mesh generation, repeated linear or nonlinear solves, and substantial computational cost, especially when many parameter sweeps or repeated queries are needed. Purely data-driven neural networks can approximate flow fields from examples, but they may produce physically inconsistent solutions and usually require dense labeled simulation data.

Physics-informed neural networks (PINNs) address a different point in this design space. Rather than replacing high-fidelity computational fluid dynamics (CFD), PINNs provide a differentiable, continuous field representation whose training objective includes governing equations, boundary conditions, and initial conditions through automatic differentiation [3, 2]. This makes them attractive for data-scarce settings, inverse or partially observed problems, and repeated-query surrogate modeling. Their practical performance, however, is limited by optimization difficulty, collocation-point allocation, imbalance between PDE, boundary-condition, and initial-condition loss terms, and implementation overhead.

This project studies these issues using the incompressible laminar cylinder-flow benchmark from Rao et al. [4]. We reproduce the mixed-variable PINN formulation, validate it against a Chorin

Preprint.

projection reference solver, and investigate training improvements along three practical dimensions: backend efficiency, adaptive collocation, and temporal loss scheduling. Our contributions are:

- We reproduce the mixed-variable PINN benchmark for $Re = 10$ incompressible cylinder flow and evaluate the learned u, v, and p fields against a Chorin projection reference solution.
- We implement and benchmark a GPU-resident L-BFGS training backend that improves wall-clock time, per-evaluation latency, estimated energy, and GPU utilization while preserving comparable accuracy.
- We evaluate causality-weighted Adam pretraining quantitatively and investigate residual-aware adaptive collocation as a preliminary sampling strategy. The completed results show that loss scheduling provides the clearest measured accuracy improvement in our experiments.

Beyond these three validated directions, we also include preliminary discussions of two exploratory extensions, alternative network architectures and a three-dimensional generalization, which we report as work in progress rather than as quantitatively validated results.

2 Related Work

Physics-informed neural networks. Raissi et al. [3] introduced PINNs as a framework for solving forward and inverse PDE problems by embedding differential-equation residuals into neural network training. Karniadakis et al. [2] surveyed PINNs as a broader scientific machine-learning paradigm. Compared with conventional supervised surrogates, PINNs reduce the need for dense labeled solution fields by using the physical residual itself as a training signal.

PINNs for incompressible flow. Rao et al. [4] proposed a mixed-variable PINN formulation for incompressible laminar flow, using a stream function and stress variables to improve constraint satisfaction. This formulation is especially useful for two-dimensional incompressible flow because the stream function enforces the continuity equation by construction. Our reproduction follows this mixed-variable formulation and uses it as the baseline for evaluating training improvements.

Training and sampling challenges. Despite their flexibility, PINNs often suffer from optimization imbalance, spectral bias, and inefficient collocation usage. Fixed random collocation may spend training budget on regions that already satisfy the PDE while under-sampling regions with larger residuals. Residual-based adaptive sampling addresses this issue by reallocating collocation points during training. Temporal causality weighting further improves transient-flow training by encouraging earlier time segments to be learned before later segments [7]. These observations motivate our focus on training strategy rather than only model architecture.

3 Problem Setup and PINN Formulation

3.1 Governing Equations and Benchmark

We consider two-dimensional incompressible laminar flow around a circular cylinder in a rectangular channel. Let $\mathbf{u} = (u, v)$ denote velocity, p pressure, ρ density, and μ dynamic viscosity. In stress form, the incompressible Navier–Stokes equations are

$$\nabla \cdot \mathbf{u} = 0, \tag{1}$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = \frac{1}{\rho}\nabla \cdot \boldsymbol{\sigma}, \tag{2}$$

$$\boldsymbol{\sigma} = -p\mathbf{I} + \mu\left(\nabla\mathbf{u} + \nabla\mathbf{u}^T\right), \tag{3}$$

where $\boldsymbol{\sigma}$ is the Cauchy stress tensor. Following the cylinder-flow setting used by Rao et al. [4] and related CFD benchmark definitions [5], we evaluate the $Re = 10$ case over $t \in [0, 0.5]$ s. Although this is a low-Reynolds-number laminar case, the benchmark is time-dependent and is useful for evaluating transient PINN training behavior.

The reference solution is generated by a Chorin fractional-step projection method [1] on a uniform 220×82 grid with time step $\Delta t = 0.001$ s. We use this reference to evaluate the learned velocity and pressure fields at multiple time snapshots. Pressure errors are reported with caution because pressure in incompressible flow is sensitive to the pressure gauge and to projection-step splitting error.

3.2 Mixed-Variable PINN

We use the mixed-variable PINN formulation from Rao et al. [4]. The neural network maps the spatiotemporal coordinate (x, y, t) to five output fields:

$$(x, y, t) \mapsto (\psi, p, \sigma_{11}, \sigma_{22}, \sigma_{12}), \tag{4}$$

where ψ is the stream function, p is pressure, and σ_{ij} are stress components. The velocity field is recovered from the stream function as

$$u = \frac{\partial \psi}{\partial y}, \qquad v = -\frac{\partial \psi}{\partial x}. \tag{5}$$

This construction satisfies incompressibility identically because

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = \frac{\partial^2 \psi}{\partial x \partial y} - \frac{\partial^2 \psi}{\partial y \partial x} = 0. \tag{6}$$

The PDE residual consists of momentum residuals, stress-constitutive residuals, and pressure–stress consistency. In compact form, the physical objective is

$$\mathcal{L} = \mathcal{L}_{\mathrm{PDE}} + \lambda_{\mathrm{wall}} \mathcal{L}_{\mathrm{wall}} + \lambda_{\mathrm{inlet}} \mathcal{L}_{\mathrm{inlet}} + \lambda_{\mathrm{outlet}} \mathcal{L}_{\mathrm{outlet}} + \lambda_{\mathrm{ic}} \mathcal{L}_{\mathrm{ic}}, \tag{7}$$

where $\mathcal{L}_{\mathrm{PDE}}$ is the mean-squared residual over collocation points and the remaining terms enforce no-slip wall conditions, inlet velocity, outlet pressure, and the initial condition. We use fixed loss weights $\lambda_{\mathrm{wall}} = \lambda_{\mathrm{inlet}} = 5$ and $\lambda_{\mathrm{outlet}} = \lambda_{\mathrm{ic}} = 1$, with unit weight on $\mathcal{L}_{\mathrm{PDE}}$. All spatial and temporal derivatives are computed by automatic differentiation.

3.3 Network and Training Configuration

The network is a fully-connected multilayer perceptron with seven hidden layers of 50 units and $\tanh$ activations, mapping the three inputs (x, y, t) to the five output fields $(\psi, p, \sigma_{11}, \sigma_{22}, \sigma_{12})$; weights are initialized with Xavier-normal initialization. The $Re = 10$ regime is set by density $\rho = 1.0$ and dynamic viscosity $\mu = 0.005$, with a pulsatile inlet of period 1.0 s over $t \in [0, 0.5]$ s. Collocation points are drawn by Latin-hypercube sampling: 100,000 points over the full domain, augmented with 15,000 points refining the near-cylinder wake region and 3,000 points along each of the two channel walls. Training uses Adam for 20,000 iterations at a learning rate of 10^{-3}, followed by an L-BFGS phase; the *reproduce* baseline runs L-BFGS through SciPy L-BFGS-B, whereas the GPU-resident and causal runs use `torch.optim.LBFGS`. For causal weighting, the time domain is split into $M = 32$ equal-duration temporal bins.

4 Training Improvements

4.1 GPU-Resident L-BFGS Backend

The reproduction baseline uses a SciPy L-BFGS-B optimization loop. This approach is straightforward but repeatedly passes flattened parameters, loss values, and gradients between the optimizer and the tensor framework. To reduce this overhead, we implement a GPU-resident training backend using `torch.optim.LBFGS`. In this implementation, model parameters, loss evaluation, and gradient computation remain on the GPU during each L-BFGS closure, reducing repeated host-device transfers. This modification changes only the execution backend; the PINN objective and evaluation protocol remain unchanged.

4.2 Residual-Aware Adaptive Collocation

[**TODO (Ming-Yang Wu)**: describe the residual-aware adaptive collocation method.]

4.3 Temporal Causality and Loss Balancing

For transient flows, treating time simply as another neural-network input may underuse the temporal ordering of the physical process. We therefore incorporate temporal-causality weighting [7]. The time domain is partitioned into M equal-duration bins (we use $M = 32$; see Section 3.3), and the loss for bin i is weighted by

$$w_i = \exp\left(-\frac{\epsilon}{M}\sum_{k<i} L_k\right), \tag{8}$$

where L_k is the residual loss of an earlier time bin and the summation is treated as a stop-gradient term. When $\epsilon = 0$, all weights become one and the objective reduces to the standard unweighted residual; the GPU-resident run optimizes exactly this unweighted objective and therefore serves as our $\epsilon = 0$ reference, so we do not run a separate $\epsilon = 0$ ablation. We evaluate two strategies: keeping causal weights fixed during L-BFGS (*frozen*) and switching back to the unweighted physical loss during L-BFGS (*uniform*).

We also consider adaptive loss weighting to reduce competition between PDE residuals and boundary-condition terms. However, preliminary gradient-norm and neural-tangent-kernel-based loss-balancing variants did not yield measurable field-L_2 improvements in our setting, and some configurations drifted toward a trivial near-zero-flow solution; we report these as negative results. In this report, the most complete quantitative results come from causality-weighted Adam pretraining and backend-efficiency experiments. Adaptive collocation, adaptive loss weighting, and architecture variants are therefore treated as partially validated directions unless additional fixed-grid L_2 results are included.

5 Results and Analysis

5.1 Evaluation Metrics

For each scalar field $q \in \{u, v, p\}$, the relative L_2 error is defined as

$$\varepsilon_q = \frac{\|q_{\text{pred}} - q_{\text{ref}}\|_2}{\|q_{\text{ref}}\|_2}. \tag{9}$$

We report errors as percentages. *Mean* L_2 is computed by averaging per-snapshot errors over all evaluated post-initial-transient snapshots with $t \geq 0.1$ s. *Global* L_2 is computed after stacking all evaluated snapshots into a single vector. These two metrics can differ because averaging per-snapshot ratios is not equivalent to taking one ratio over the full trajectory. For pressure, both predicted and reference fields are spatially demeaned at each snapshot before computing ε_p, removing the arbitrary additive pressure gauge.

5.2 Baseline Reproduction and Backend Efficiency

The reproduction baseline and the GPU-resident backend reach comparable post-initial-transient accuracy (mean L_2 errors summarized in Table 2): the GPU-resident run shows slightly larger velocity errors but a slightly smaller mean pressure error. Pressure is the least stable field—for example, near $t = 0.5$ s the per-snapshot pressure error rises above 60%—reflecting the gauge sensitivity of projection-based incompressible-flow references.

Table 1 shows that the GPU-resident backend improves computational efficiency. For 1000 L-BFGS function evaluations on one RTX 3090, it achieves a $1.23\times$ speedup, 18.6% lower per-evaluation latency, 11.0% lower estimated energy, and 14 percentage points higher mean GPU utilization.

5.3 Adaptive Collocation Behavior

[**TODO (Ming-Yang Wu)**: report the adaptive-collocation results.]

5.4 Causal Training Performance

Table 2 compares the reproduction baseline, GPU-resident backend, and causal training variants. The "Loss" column is the final training objective value for each run and should be interpreted to-

Table 1: Backend benchmark: 1000 L-BFGS function evaluations on one RTX 3090. Energy is estimated as mean GPU power draw multiplied by wall-clock time.

Metric	reproduce	GPU-resident	Change
Wall time	204.1 s	**166.4** s	1.23× faster
Eval latency	204 ms	**166** ms	18.6% lower
Energy	59.8 kJ	**53.2** kJ	11.0% lower
GPU utilization	68%	**82**%	+14 pp

gether with the L_2 metrics because causal and uniform phases use different loss schedules. Causality-weighted Adam pretraining followed by an unweighted L-BFGS polish achieves the lowest error among the tested configurations. The best configuration is causal $\epsilon = 30$ with unweighted L-BFGS polish, which achieves mean relative L_2 errors of 4.64%, 18.60%, and 8.89% for u, v, and p. Figure 2 shows the predicted and reference u, v, and p fields for this configuration at $t = 0.5$ s; the velocity fields match closely and the demeaned pressure recovers the correct large-scale structure.

Table 2: $Re = 10$ post-initial-transient error ($t \geq 0.1$ s). *frozen* keeps causal weights during L-BFGS; *uniform* switches back to the unweighted physical loss during L-BFGS.

		Mean L_2 (%)			Global L_2 (%)		
Run	Loss	ε_u	ε_v	ε_p	ε_u	ε_v	ε_p
reproduce (baseline)	2.10×10^{-4}	5.30	19.86	10.79	4.47	14.75	27.92
GPU-resident	2.94×10^{-3}	5.94	22.23	10.31	4.75	16.44	24.42
causal ϵ=30, frozen	2.31×10^{-4}	5.22	20.38	11.03	4.36	15.02	28.49
causal ϵ=30, uniform	$\mathbf{1.53\times10^{-4}}$	**4.64**	**18.60**	**8.89**	**3.93**	**13.28**	**22.51**
causal ϵ=50, frozen	2.11×10^{-4}	5.34	19.86	10.10	4.47	15.05	25.83
causal ϵ=50, uniform	1.54×10^{-4}	4.96	19.21	9.70	4.14	13.93	25.30

The comparison between *frozen* and *uniform* L-BFGS suggests that causal weighting is most useful as an initialization mechanism during Adam training. Keeping causal weights frozen during L-BFGS can bias the optimizer toward earlier time bins, while switching back to the unweighted physical loss allows L-BFGS to polish the full trajectory. This supports a two-stage interpretation: causal weighting improves the training path, but the final model should be optimized against the original unweighted physical objective. As shown in Figure 1, the minimum causal weight $\min_i w_i$ rises gradually over Adam training but saturates near 0.84 ($\epsilon = 30$) and 0.77 ($\epsilon = 50$), indicating that the curriculum does not fully reach the latest time bins within the iteration budget; this is consistent with $\epsilon = 30$ and $\epsilon = 50$ converging to nearly identical accuracy. The corresponding unweighted training-loss trajectories are shown in Figure 3.

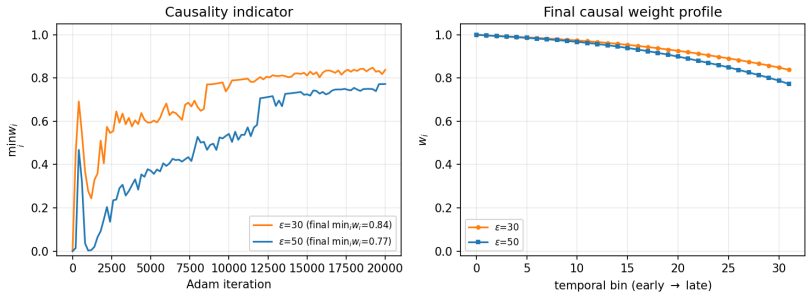

Figure 1: Causal curriculum. *Left*: minimum bin weight $\min_i w_i$ versus Adam iteration, saturating at 0.84 (ϵ=30) and 0.77 (ϵ=50) without reaching unity, so the curriculum never fully propagates to the latest bins. *Right*: final weight profiles decrease monotonically from early to late bins, with ϵ=50 down-weighting late bins more.

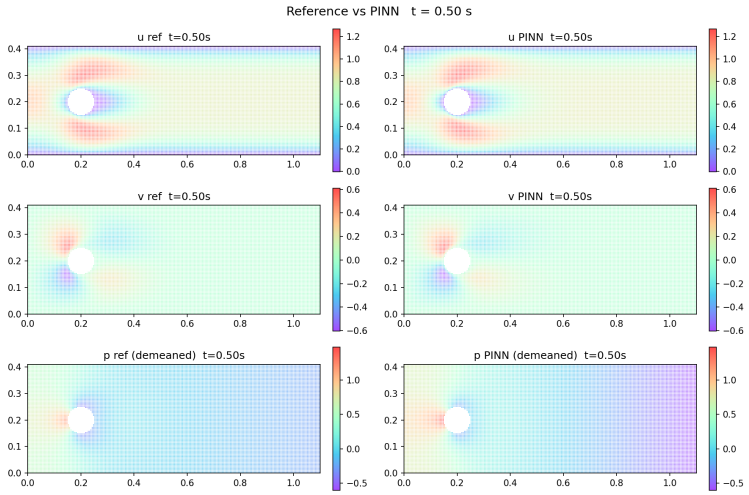

Figure 2: Reference versus PINN fields at $t = 0.5$ s for the best configuration (causal ϵ=30, unweighted L-BFGS polish); rows show u, v, and demeaned p. Velocity agrees closely; demeaned pressure recovers the large-scale structure but is the least accurate field.

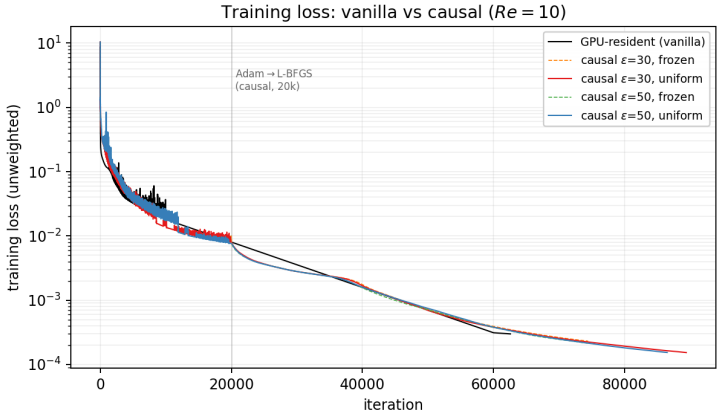

Figure 3: Unweighted training loss versus iteration ($Re = 10$, log scale) for the GPU-resident baseline and the four causal runs, all using the Adam→L-BFGS schedule (Adam ends at iteration 20,000). The ϵ=30 and ϵ=50 runs nearly coincide; the uniform-polish variants (solid) reach the lowest final loss.

5.5 Model Architecture Variants

[**TODO (Chen-Yan Juang)**: discuss the model architecture variants explored, such as SIREN activations [6].]

5.6 3D Extension

[**TODO (Jing-Hua Chang)**: discuss the extension from the 2D formulation to 3D obstacle flow.]

6 Conclusion

We reproduced the mixed-variable PINN benchmark for $Re = 10$ incompressible flow around a cylinder and obtained comparable post-initial-transient accuracy against a Chorin projection reference solution. Our GPU-resident L-BFGS backend reduced wall-clock time by $1.23\times$ and lowered estimated energy by 11.0% while preserving similar accuracy. Beyond backend efficiency, causality-

weighted Adam pretraining followed by unweighted L-BFGS polish achieved the best mean relative L_2 errors among our tested configurations (4.64%, 18.60%, and 8.89% for u, v, and p), indicating that temporal loss scheduling can improve PINN optimization.

The main takeaway is that PINN performance for unsteady laminar flow is not determined by the model architecture alone. Backend execution and loss scheduling already show measurable effects on efficiency and accuracy, while adaptive collocation remains a promising sampling mechanism that should be validated with fixed-grid L_2 metrics. Future work should integrate adaptive collocation, adaptive loss weighting, and architecture variants into a unified model, and extend the evaluation to higher Reynolds numbers and more complex geometries, building on the preliminary three-dimensional extension outlined in Section 5.6.

7 Code and Reproducibility

The full codebase is available at `https://github.com/inQuitiVe/ECE228_PINN`. The main reproduction experiments use the $Re = 10$ cylinder-flow benchmark, a Chorin projection reference solution, Adam initialization followed by L-BFGS optimization, and the evaluation metrics defined in Section 5.1. The backend benchmark was performed on one RTX 3090 using 1000 controlled L-BFGS function evaluations.

8 Team Member Contribution

- **Chi-Han Chiu**: Implemented causal training, GPU-resident L-BFGS benchmarking, core PINN reproduction, and quantitative evaluation.
- **Chen-Yan Juang**: Investigated model architecture variants, including SIREN-style activations, for improving field representation.
- **Ming-Yang Wu**: Developed and tested residual-aware adaptive collocation strategies for sampling.
- **Jing-Hua Chang**: Explored the extension from the baseline 2D formulation to 3D obstacle-flow settings.

References

[1] Alexandre Joel Chorin. Numerical solution of the Navier–Stokes equations. *Mathematics of Computation*, 22(104):745–762, 1968.

[2] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.

[3] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[4] Chengping Rao, Hao Sun, and Yang Liu. Physics-informed deep learning for incompressible laminar flows. *Theoretical and Applied Mechanics Letters*, 10(3):207–212, 2020. arXiv:2002.10558.

[5] Michael Schäfer, Stefan Turek, Franz Durst, Egon Krause, and Rolf Rannacher. Benchmark computations of laminar flow around a cylinder. In *Flow Simulation with High-Performance Computers II*, volume 48 of *Notes on Numerical Fluid Mechanics (NNFM)*, pages 547–566. Vieweg+Teubner Verlag, 1996.

[6] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.

[7] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024.